



Freque README v2.4

Freque Plugin Documentation

v2.4

Complete documentation for audio-reactive visualizations with 2D Canvas, WebGL/WebGL2 shaders, advanced Three.js techniques, and procedural skeletal animation



Documentation Package

Total Size: ~125 KB

Files: 9 comprehensive guides

Version: 2.4 (November 14, 2025)

What's New in v2.4

GLB/GLTF Character Integration Guide

NEW CAPABILITY: Complete implementation guide for audio-reactive character animation!

-  **GLB/GLTF Model Loading** - Load and integrate character models
-  **Procedural Animation Patterns** - Real-time bone rotation systems
-  **Multi-Cycle Rhythm Layering** - Organic, non-repetitive movement
-  **Relative Transformations** - Preserve base animation poses
-  **Audio-Reactive Systems** - Beat detection and energy mapping
-  **Generic Patterns** - Applicable to any animation style
-  **Complete Code Examples** - Ready-to-adapt implementations

Example Use Cases: Any character animation style - dancing, gesturing, idle motion, reactive poses, etc.

UI Enhancement: Custom Dial Fill Colors

NEW FEATURE: Plugins can now specify custom dial fill colors using CSS variables!

-  **Theme-Aware Colors** - Use any CSS variable (e.g., `--accent-color`)
-  **Automatic Theme Support** - Colors change with theme selection
-  **Simple API** - One property in plugin config: `dialFillColor: '--accent-color'`
-  **Visual Consistency** - Easy to identify plugin dials

Example:

```
super('myplugin', visualizer, {
  dialFillColor: '--accent-color' // All dials use accent color
});
```

Documentation Updates

- **Character Integration Guide** → NEW (32 KB) - GLB/GLTF character animation guide
 - **v2.4 Summary** → NEW (7 KB) - Update overview and implementation details
 - **Development Guide** → Updated with dial fill color documentation
 - **Claude Guide** → Updated with dial color prompting examples
 - **Quick Reference Card** → Updated to v2.4 with dial color patterns
-

What's New in v2.3

UI Controls Update: Dials & Standardized Toggles

Control System Improvements:

-  **Dial Controls** - Rotary knobs replace sliders for professional feel
-  **Standardized Toggle Styling** - All toggles use `className: 'btn-primary-mixer'`
-  **Visual Consistency** - Matches professional audio/video software UX
-  **100% Backwards Compatible** - Old code still works

Changes: - All numeric controls now use `type: 'dial'` instead of `type: 'slider'`
- All toggle controls must include `className: 'btn-primary-mixer'` - Integration layer respects custom `className` config

Example:

```
// Dial control (replaces slider)
this.addControl('speed', {
  type: 'dial',
  label: 'Speed',
  min: 0,
  max: 3,
  step: 0.1,
  value: 1.0,
  onChange: (value) => { this.speed = value; }
});

// Toggle control (with required className)
this.addControl('audioReactive', {
  type: 'checkbox',
  label: 'Audio Reactive',
  checked: false,
  className: 'btn-primary-mixer',
  onChange: (value) => { this.audioReactive = value; }
});
```

What's New in v2.2

Major Update: WebGL/WebGL2 Plugin Support

NEW CAPABILITY: Raw WebGL and WebGL2 shader-based plugins are now fully supported!

-  **WebGL2 Shader Plugins** - Create advanced procedural effects
-  **WebGL 1.0 Plugins** - Maximum browser compatibility
-  **Context Flexibility** - Plugins choose 2D, WebGL, or WebGL2
-  **Lazy Context Loading** - Base class no longer forces 2D context
-  **100% Backwards Compatible** - All existing plugins work unchanged

Example: The new PSYCH plugin uses WebGL2 for procedural liquid bubble effects running at 60fps in 4K.

Documentation Updates

- **Development Guide** → v2.2 (38 KB) - Added WebGL/WebGL2 sections
 - **Claude Guide** → v2.2 (28 KB) - Added shader plugin prompting
 - **v2.2 Summary** → New (12 KB) - Architecture change details
-

Documentation Files

1 [README.md](#) ★ **START HERE**

This file - Complete navigation guide

2 [DOCUMENTATION_UPDATES_V2_4_SUMMARY.md](#) (7 KB) ★ **NEW**

What's new in v2.4 and overview of procedural animation guide

Contents: - What's New in v2.4 summary - Documentation structure and organization - Procedural skeletal animation overview - Implementation patterns (relative transforms, multi-cycle, feet planting) - Audio integration patterns - Common pitfalls and solutions - Migration notes (100% backwards compatible) - Support and resources

Who should read this: - Anyone wanting a quick v2.4 overview - Developers considering skeletal animation plugins - Technical leads evaluating new capabilities

3 [GLTF_CHARACTER_PLUGIN_INTEGRATION_GUIDE.md](#) (32 KB) ★ **NEW**

Complete guide to GLB/GLTF character animation in Three.js

Contents: - Core concepts (procedural vs pre-baked animations) - Implementation architecture (bone systems, state machine) - Critical technical patterns (relative transforms, feet planting, multi-cycle layering) - Audio reactivity (energy mapping, beat detection, state transitions) - UI controls (mode toggles, sensitivity, positioning) - Common pitfalls & solutions (character rotation, feet movement, repetition) - Complete code examples (full procedural method, integration, bone extraction) - Best practices for skeletal animation - Performance considerations - Testing checklist - Prompting Claude for character plugins

Who should read this: - Developers creating GLB/GLTF character plugins - Anyone implementing procedural bone manipulation - Those needing audio-reactive animation systems - Troubleshooting skeletal animation issues

4 [DOCUMENTATION_UPDATES_V2_2_SUMMARY.md](#) (12 KB)

What changed in v2.2 and why

Contents: - Executive summary of architecture changes - Before/after comparison - Impact on existing plugins (none!) - New plugin types supported (WebGL/WebGL2) - Developer guidelines by plugin type - PSYCH plugin example - Testing recommendations

Who should read this: - Developers wanting to understand the v2.2 changes - Technical leads evaluating the new architecture - Anyone creating WebGL/WebGL2 plugins

5 [FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md](#) (38 KB) ★ **TECHNICAL REFERENCE**

The complete technical reference for plugin developers

NEW in v2.2: -  Canvas Context Types section -  2D Canvas Plugins section -  WebGL/WebGL2 Plugins section (complete templates) -  Three.js context handling notes -  Plugin type decision guide -  Context type compatibility matrix

Contents: - Quick Start (basic plugin template) - **Canvas Context Types** ★ NEW - **2D Canvas Plugins** ★ NEW - **WebGL/WebGL2 Plugins** ★ NEW (complete templates) - **Three.js Integration** - Cube Camera System for Video Reflections - Layer Management - Color Space Management - Dual Mapping Modes - Audio Integration - UI Controls - Performance Optimization - Best Practices

Who should read this: - Developers manually coding plugins - Anyone implementing WebGL/WebGL2 shaders - Technical leads architecting new plugins

6 [CREATING_PLUGINS_WITH_CLAUDE_V2_2.md](#) (28 KB) ★ FOR CLAUDE

Guide for using Claude to create Freque plugins

NEW in v2.2: - ✓ WebGL/WebGL2 plugin prompting - ✓ Procedural shader request patterns - ✓ 2D Canvas plugin templates - ✓ WebGL troubleshooting section - ✓ PSYCH-style advanced requests

Contents: - Quick Start (10-minute first plugin) - The Universal Plugin Prompt Template - **Requesting 2D Canvas Plugins** ★ NEW - **Requesting WebGL/WebGL2 Plugins** ★ NEW (shader effects) - Requesting Three.js Plugins - Video Reflection Systems - Common Request Patterns - **Troubleshooting** (expanded with WebGL) - Visual Description Library - Real-World Examples

Who should read this: - Non-technical users wanting to create plugins - Technical users who want to use Claude for faster development - Anyone learning effective prompting for code generation

7 [CUBE_CAMERA_QUICK_REFERENCE_V2_1.md](#) (11 KB)

One-page quick reference for cube camera implementation

Contents: - When to Use Cube Cameras - 5-Step Implementation (copy-paste ready) - Advanced: Dynamic Positioning - Layer System - Performance Optimization - Common Issues & Solutions - Complete Minimal Example - Asking Claude Template

Who should read this: - Developers implementing cube cameras RIGHT NOW - Quick reference during coding - Troubleshooting guide

[CHROMOSPHERES_ANALYSIS.md](#) (12 KB)

Deep dive into Chrome Spheres v1.2.0 → v1.3.0 changes

Contents: - Executive Summary - Major Architectural Changes - Cube Camera System Implementation - Dynamic Positioning - Background Sphere Visibility - Dual Texture System - Color Space Management - Enhanced Quality Settings - Performance Optimizations - Architectural Lessons Learned

Who should read this: - Developers wanting to understand cube camera architecture - Anyone curious about the technical evolution - Technical leads evaluating the approach

[DOCUMENTATION_UPDATES_V2_1_SUMMARY.md](#) (11 KB)

What changed in v2.1 (previous update)

Contents: - Float speed non-linear scaling - Video intensity auto-adjustment - Control update behavior - Minor clarifications

Quick Navigation Guide

I want to...

...understand what's new in v2.4 → Read [DOCUMENTATION_UPDATES_V2_4_SUMMARY.md](#)

...implement GLB/GLTF character animation → Read [GLTF_CHARACTER_PLUGIN_INTEGRATION_GUIDE.md](#) → See example character plugins for working implementations

...understand what's new in v2.2 → Read
[DOCUMENTATION_UPDATES_V2_2_SUMMARY.md](#)

...create a WebGL/WebGL2 shader plugin → Read
[CREATING_PLUGINS_WITH_CLAUDE_V2_2.md](#) - WebGL sections → Reference
[FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md](#) - WebGL templates

...implement cube cameras myself → Read
[FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md](#) Cube Camera section → Use
[CUBE_CAMERA_QUICK_REFERENCE_V2_1.md](#) during coding

...use Claude to create a plugin with video reflections → Read
[CREATING_PLUGINS_WITH_CLAUDE_V2_2.md](#) Video Reflection section

...understand why Chrome Spheres works the way it does → Read
[CHROMOSPHERES_ANALYSIS.md](#)

...see all available documentation → This README or [INDEX.md](#)

Key Concepts

Plugin Types (v2.2)

Type	When to Use	Browser Support	Example
2D Canvas	Simple visualizations, drawing API	100%	Waveforms, basic particles
WebGL 1.0	GPU effects, max compatibility	~97%	GPU particles, basic shaders
WebGL2	Advanced shaders, modern features	~95%	PSYCH , procedural effects
Three.js	3D scenes, video reflections	~97%	Chrome Spheres, 3D objects

Canvas Context Architecture (v2.2)

The Problem: A canvas can only have ONE context type. Once you create a 2D context, you can't get a WebGL context.

The Solution: The base class now uses lazy context loading. It doesn't create any context until needed, allowing plugins to choose their type.

Result: WebGL/WebGL2 plugins now work! 🎉

Backwards Compatibility

✅ All existing 2D plugins work unchanged ✅ All existing Three.js plugins work unchanged ✅ New: WebGL/WebGL2 plugins now possible



Getting Started Paths

Path 1: Non-Technical User (Create with Claude)

1. Read “Quick Start” in [CREATING_PLUGINS_WITH_CLAUDE_V2_2.md](#)
2. Choose plugin type (2D/WebGL/Three.js)
3. Copy the appropriate template
4. Fill in your vision
5. Send to Claude
6. Save and test the result

Time: 15 minutes to first plugin

Path 2: Developer Creating WebGL/WebGL2 Plugin

1. Read “Canvas Context Types” in [FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md](#)
2. Read “WebGL/WebGL2 Plugins” section
3. Copy the appropriate template (WebGL or WebGL2)
4. Implement shaders and uniforms
5. Test at different resolutions
6. Optimize for 60fps

Time: 2-4 hours for first WebGL plugin

Path 3: Developer Adding Video Reflections (Three.js)

1. Scan [DOCUMENTATION_UPDATES_V2_2_SUMMARY.md](#) for context
2. Read Cube Camera System section in [FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md](#)
3. Use [CUBE_CAMERA_QUICK_REFERENCE_V2_1.md](#) during implementation

4. Study Chrome Spheres plugin code for details
5. Test performance and optimize

Time: 1-2 hours for first implementation

Path 4: Technical Lead Evaluating v2.2

1. Read [DOCUMENTATION_UPDATES_V2_2_SUMMARY.md](#) executive summary
2. Review architectural changes (lazy context loading)
3. Verify backwards compatibility claims
4. Review WebGL plugin templates
5. Consider implications for plugin ecosystem

Time: 30 minutes for evaluation



Package Statistics

File Count by Version

Version	Files	Added	Total Size
v2.0	5	-	~82 KB
v2.1	7	+2	~105 KB
v2.2	9	+2	~120 KB

Content Metrics

Metric	v2.1	v2.2	Added
Word Count	57,200	62,500	+5,300
Code Examples	106	125	+19
Plugin Templates	3	6	+3
Troubleshooting	8	13	+5



Version History

v2.2 (Current Session) - WebGL/WebGL2 Support

- ☒ Added WebGL/WebGL2 plugin architecture
- ☒ Added context type flexibility
- ☒ Updated Development Guide with WebGL sections
- ☒ Updated Claude Guide with shader prompting
- ☒ 100% backwards compatible
- ☒ Complete templates for all plugin types

v2.1 (November 9, 2025)

- ☒ Added float speed scaling documentation
- ☒ Added video intensity adjustment
- ☒ Added control update behavior
- ☒ Added minor clarifications
- ☒ 100% implementation coverage

v2.0 (November 8, 2025)

-  Added cube camera system
-  Added layer management
-  Added color space management
-  Added dynamic positioning
-  95% implementation coverage

v1.0 (Previous)

-  Basic Three.js integration
-  Standard patterns
-  Audio integration
-  UI controls

Supported Plugin Types

Before v2.2

 2D Canvas plugins  Three.js plugins (WebGL internally)  Raw WebGL plugins (BROKEN)  Raw WebGL2 plugins (BROKEN)

After v2.2

 2D Canvas plugins (unchanged)  WebGL 1.0 plugins (NOW WORKS)  
WebGL2 plugins (NOW WORKS)   Three.js plugins (unchanged)

Real-World Examples

2D Canvas

- Waveform displays

- Spectrum analyzers
- Simple particle effects
- Text visualizations

WebGL 1.0

- GPU-accelerated particle systems (10,000+ particles)
- Basic shader effects
- Maximum compatibility

WebGL2

- **PSYCH** - Procedural liquid bubble shader 
- Advanced noise-based effects
- Complex fragment shaders
- Raymarching/SDF techniques

Three.js

- **Chrome Spheres** - 3D with cube cameras
- 3D object visualizations
- Environment mapping
- Video reflections

Success Metrics

After reading this documentation, you should be able to:

✓ Choose the right plugin type for your visualization ✓ Explain why WebGL/ WebGL2 plugins are now possible ✓ Create 2D, WebGL, or Three.js plugins ✓ Request shader-based plugins from Claude ✓ Implement cube cameras for video reflections ✓ Optimize for 60fps at 4K resolution ✓ Troubleshoot common WebGL issues ✓ Understand the architecture changes in v2.2

Support

Questions About WebGL/WebGL2?

1. Check Development Guide WebGL sections
2. Review Claude Guide shader prompting
3. Study troubleshooting section
4. Ask Claude with specific context

Questions About Implementation?

1. Check Quick Reference first
2. Review relevant section in Development Guide
3. Study example plugin code
4. Ask Claude with specific context

Questions About Prompting?

1. Review Claude Guide examples
2. Use provided templates
3. Start simple and iterate
4. Include performance requirements

Conclusion

This documentation package provides everything needed to create **any type** of Freque plugin: 2D canvas effects, GPU-accelerated WebGL shaders, advanced WebGL2 procedural art, or sophisticated Three.js 3D scenes with video reflections.

Key Resources: - **Learning:** Start with this README - **Technical:** FREQUE_PLUGIN_DEVELOPMENT_GUIDE_V2_2.md - **Prompting:**

CREATING_PLUGINS_WITH_CLAUDE_V2_2.md - **Quick Impl:**
CUBE_CAMERA_QUICK_REFERENCE_V2_1.md - **Architecture:**
DOCUMENTATION_UPDATES_V2_2_SUMMARY.md

Total Package v2.2: ~120 KB of comprehensive, production-ready documentation

NEW in v2.2: Full WebGL/WebGL2 support unlocks a whole new category of GPU-accelerated shader visualizations! 🚀

Happy plugin development! 🚀🎨🎵

*Documentation v2.2 - WebGL/WebGL2 Enabled! Last Updated: Current Session
Based on: Freque Plugin Architecture Update*

Freque Plugin Development Documentation

© 2025 Freque - Professional Audio Visualization Platform